

Supplementary Material

A Impact Statement

Platonic Alignment: Escaping the Plato’s Cave.

Platonic Representation Hypothesis [?] suggests that vision and language models, though trained on disjoint objectives and data, encode latent geometries that are mutually compatible. Our work turns this philosophical and observational claim into practice: we *escape Plato’s Cave* by *surfacing shared structure from representations originally confined to separate unimodal worlds*, while keeping every pretrained unimodal backbones completely frozen.

Joint Autoencoder Modulator (JAM).

We propose JAM, a post-hoc adaptor for platonic alignment that:

- *Preserves specialization* – Modality-specific autoencoders reconstruct each latent space, guarding task-relevant features.
- *Enforces cross-modal coherence* – Our novel spread loss couples the JAM framework, amplifying the latent commonality predicted by the Platonic hypothesis.
- *Light-weight framework* – no end-to-end multimodal fine-tuning (nor pre-training); refer to D.2 that shows the lightweight structure of JAM.

Contributions.

- *Platonic Alignment through Specialist Task Settings*: We test the platonic alignment through both statistical and model-training framework in specialist task settings which require more fine-grained contextual understanding of the data.
- *Revisiting multimodal training pipeline strategy*: Our results show that aligning the frozen pretrained backbones can rival with inherently multimodal models, charting a potential alternative strategy for creating multimodal models for specialized task frameworks.

B Statistical Metrics to Measure Platonic Alignment

B.1 Thematic analysis of statistical representation alignment tests

We focus on the following four statistical metrics for testing Platonic alignment: Canonical Correlation Analysis (CCA [?]) with linear and kernel (rbf kernel) PCA, Singular Value CCA (SVCCA [?]), Centered Kernel Alignment (CKA [?]), and Centered Kernel Nearest Neighbors Alignment (CKNNA [? ?]).

The following are the thematic analyses that demonstrate our metrics selection strategy:

- **Unsupervised and second-order**: All four metrics are *unsupervised* and based on second-order statistics (e.g., inner products or covariances), making them scale-invariant and naturally comparable across modalities.
- **Symmetry**: The symmetry property indicates that the metric treats the data points interchangeably, meaning $d(x, y) = d(y, x)$.
- **Data-Driven (D) vs. Canonical (C)**:
 - **Data-Driven** metrics (CCA, SVCCA) learn linear projections from the data that actively mold one representation space to align with another. They measure the *best-case alignment* achievable through adaptation.
 - **Canonical** metrics (CKA, CKNNA) keep the input representations fixed and evaluate the *existing alignment structure* as is, without transformation. They quantify inherent similarity without retraining or reprojecting.
- **Global (G) vs. Local (L)**:
 - **Global** metrics (CCA, SVCCA, CKA) aggregate information across *all sample pairs*, capturing holistic alignment patterns between entire distributions.
 - **Local** metrics (CKNNA) focus on *localized structure* by comparing only mutual k -nearest neighbors, revealing fine-grained neighborhood-level similarity that may be obscured globally.

Table 1: Comparative analysis of statistical alignment metrics. All metrics are unsupervised and based on second-order statistics.

Metric	Property				Description
	Symmetric	Data-Driven(D) vs. Canonical(C)	Global(G) vs. Local(L)	Batchable	
CCA	✓	D	G	✓	Learns projections to maximize cross-view correlation. Sensitive to high-dimensional data structure, hence apply linear or kernel PCA before applying CCA.
SVCCA	✓	D	G	✓	Applies SVD to smooth and compress representations before applying CCA.
CKA	✓	C	G	✓	Compares original representations using kernel alignment, invariant to rotation and scale.
CKNNA	✓	C	L	✓	A local variant of CKA restricted to shared k -nearest neighbors; reveals localized alignment structure.

- **Batchable:** A computational property that makes it feasible to compute in a reasonable time frame.

B.2 Metrics Formulation

Recall our glossary:

Table 2: Glossary of variables and symbols

Symbol	Description
Φ_V, Ψ_V	Vision (image) autoencoder (encoder, decoder)
Φ_L, Ψ_L	Language (text) autoencoder (encoder, decoder)
V	Set of image embeddings ($ V = n, v_i \in V, i = \{1, \dots, n\}$)
L	Set of text embedding (i.e., $L_P \cup L_N; L = 2n$)
L_P	Set of positive text embeddings ($ L_P = n, l_{P_i} \in L_P$)
L_N	Set of hard negative text embeddings ($ L_N = n, l_{N_i} \in L_N$)
d_{input}, d_{latent}	Input, Latent (i.e., bottleneck) dimension of autoencoder
l_i	Element of $L \setminus \{l_{P_i}\}$ with anchor index i
$C_L(i)$	Set of similar context text embeddings
$\bar{C}_L(i)$	Set of dissimilar context text embeddings (i.e., $L \setminus C_L(i)$)

Kernel preliminaries Let $\{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^n$ be paired samples: in our case, let x_i be the images and y_i be the corresponding texts. A representation function $f_V : \mathcal{X} \rightarrow V \in \mathbb{R}^d$ maps inputs to feature vectors $v_i = f_V(\mathbf{x}_i)$ in an RKHS (Reproducing Kernel Hilbert Space) $\mathcal{H}$ [?] equipped with the inner-product kernel $K_{ij} = \kappa(v_i, v_j) = \langle v_i, v_j \rangle$. Analogously f_L maps $\mathbf{y}_i$ to l_i with kernel $L_{ij} = \langle l_i, l_j \rangle$. Center both kernels to remove mean effects via the centring matrix $H = I - \frac{1}{n} \mathbf{1}\mathbf{1}^\top$:

$$\bar{K} = HKH, \quad \bar{L} = HLH.$$

B.2.1 Centered-Kernel Alignment (CKA)

CKA [?] normalises the *Hilbert–Schmidt Independence Criterion* (HSIC) [?] between the centred kernels:

$$\text{HSIC}(K, L) = \frac{1}{(n-1)^2} \text{tr}(\bar{K} \bar{L}), \quad (1)$$

$$\text{CKA}(K, L) = \frac{\text{HSIC}(K, L)}{\sqrt{\text{HSIC}(K, K) \text{HSIC}(L, L)}}. \quad (2)$$

Expanding the trace in Equation 1 shows that every pair (i, j) contributes a product of centered similarities:

$$\text{tr}(\bar{K} \bar{L}) = \sum_{i=1}^n \sum_{j=1}^n (\langle v_i, v_j \rangle - \mathbb{E}_\ell \langle v_i, v_\ell \rangle) (\langle l_i, l_j \rangle - \mathbb{E}_\ell \langle l_i, l_\ell \rangle). \quad (3)$$

Because the denominator applies the same operation to each modality, CKA is scale-invariant and bounded in $[0, 1]$; it therefore has advantage in measuring *global* geometric correspondence. However, Equation 2 aggregates *all* pairwise relations, so even a small region of mis-aligned samples suppresses the score which CKA can be too strict for measuring cross-modal cases.

B.2.2 Centered-Kernel k -NN Alignment (CKNNA)

To focus on *local* structure, the authors in [?] suggest a binary mask $\alpha(i, j)$ that selects only pairs that are *mutual* k -nearest neighbours in *both* modalities:

$$\alpha(i, j) = \mathbb{1}[v_j \in \text{kNN}(v_i) \wedge l_j \in \text{kNN}(l_i) \wedge i \neq j]. \quad (4)$$

Using this mask, one can restrict the definition of alignment to bias more towards the local structure through replacing the full HSIC trace by a *masked* cross-covariance.:

$$\text{Align}_{\text{local}}(K, L) = \sum_{i=1}^n \sum_{j=1}^n \alpha(i, j) (\langle v_i, v_j \rangle - \mathbb{E}_{\ell} \langle v_i, v_{\ell} \rangle) (\langle l_i, l_j \rangle - \mathbb{E}_{\ell} \langle l_i, l_{\ell} \rangle) \quad (5)$$

$$= \sum_{i=1}^n \sum_{j=1}^n \alpha(i, j) \bar{K}_{ij} \bar{L}_{ij}. \quad (6)$$

The final formulation of CKNNA becomes:

$$\text{CKNNA}(K, L) = \frac{\text{Align}_{\text{local}}(K, L)}{\sqrt{\text{Align}_{\text{local}}(K, K) \text{Align}_{\text{local}}(L, L)}}. \quad (7)$$

Note that CKNNA can be fully recovered to CKA for $k \rightarrow n$, since $\alpha(i, j) = 1$ for all $i \neq j$.

Thus, CKA asks: “Are the two representations **globally** linearly related?”; whereas CKNNA captures: “Do the two models agree **locally** (i.e., on each point’s neighbours)?”.

B.2.3 CCA (Canonical Correlation Analysis)

Let $X \in \mathbb{R}^{d_x \times n}$ and $Y \in \mathbb{R}^{d_y \times n}$ be column-wise zero-centered feature matrices produced by two representations (all means are removed so that covariance estimates are unbiased). Define sample covariance blocks:

$$\Sigma_{xx} = \frac{1}{n} X X^{\top}, \quad \Sigma_{yy} = \frac{1}{n} Y Y^{\top}, \quad \Sigma_{xy} = \frac{1}{n} X Y^{\top}. \quad (8)$$

CCA searches for weight vectors $\mathbf{w} \in \mathbb{R}^{d_x}$, $\mathbf{v} \in \mathbb{R}^{d_y}$ that maximise the *Pearson correlation* between projected variables:

$$\rho = \max_{\mathbf{w}, \mathbf{v}} \frac{\mathbf{w}^{\top} \Sigma_{xy} \mathbf{v}}{\sqrt{\mathbf{w}^{\top} \Sigma_{xx} \mathbf{w}} \sqrt{\mathbf{v}^{\top} \Sigma_{yy} \mathbf{v}}} \quad (9)$$

$$\text{s.t. } \mathbf{w}^{\top} \Sigma_{xx} \mathbf{w} = 1, \quad \mathbf{v}^{\top} \Sigma_{yy} \mathbf{v} = 1. \quad (10)$$

Using Lagrange multipliers we obtain the generalized eigenvalue problem:

$$(\Sigma_{xx}^{-1} \Sigma_{xy} \Sigma_{yy}^{-1} \Sigma_{yx}) \mathbf{w} = \rho^2 \mathbf{w}, \quad \mathbf{v} = \rho^{-1} \Sigma_{yy}^{-1} \Sigma_{yx} \mathbf{w}, \quad (11)$$

whose top k eigenvalues $\rho_1 \geq \rho_2 \geq \dots \geq \rho_k$ are the **canonical correlations**. We report the maximum canonical correlation value (obtained from the first canonical component) as an alignment score.

PCA-assisted CCA Since CCA method suffers from high-dimensionality data structures [?], we first project each feature vectors onto its leading r principal components:

$$\tilde{X} = P_x^{\top} X, \quad \tilde{Y} = P_y^{\top} Y, \quad (12)$$

with $P_x \in \mathbb{R}^{d_x \times r}$ (orthonormal) obtained from linear PCA or kernel PCA using an RBF kernel $\kappa(\mathbf{z}, \mathbf{z}') = \exp[-\gamma \|\mathbf{z} - \mathbf{z}'\|^2]$. CCA is then run on $\tilde{X}, \tilde{Y}$, producing more stable correlations in high-dimensional regimes. In our experiments, we used $r = 50$ (i.e., feature dimension = 50), and $k = 50$ (i.e., number of canonical components = 50).

B.2.4 SVCCA (Singular Vector Canonical Correlation Analysis)

SVCCA [?] replaces the heuristic PCA cut-off with a *data-dependent approach* via SVD:

1. Compute SVDs: $X = U_x \Sigma_x V_x^\top$, $Y = U_y \Sigma_y V_y^\top$.
2. Retain the top r_x, r_y singular vectors explaining at least a fixed proportion η (e.g., 99%) of variance:
$$\hat{X} = \Sigma_{x,[1:r_x]} V_{x,[1:r_x]}^\top, \quad \hat{Y} = \Sigma_{y,[1:r_y]} V_{y,[1:r_y]}^\top.$$
3. Run linear CCA on $(\hat{X}, \hat{Y})$ to obtain canonical correlations $\{\rho_i\}_{i=1}^k$ with $k = \min(r_x, r_y)$.

The SVCCA score is reported using mean of these k correlations, $\text{SVCCA} = \frac{1}{k} \sum_{i=1}^k \rho_i$, where we report SVCCA score with $k = 10$.

B.3 Supplemental Results for Statistical Alignment Tests

We describe the specific data pair set-up and provide results of statistical alignment tests for each specific task (i.e., Replace, Add, and Swap cases) in Sugarcrepe data [?]. In our experiments, we reduced features dimension to 50 for CCA using linear and kernel PCA; for SVCCA, CKA, and CKNNA, the features dimension was set to 10, and nearest neighbors $k = 10$ (same setting utilized in [?]).

B.3.1 Replace Task

- Match: V and L that are matching (i.e., correct correspondence)
- Easy Non-Match: V and L that have clear non-matching aspects
 - Case 1: V = White-noise image embeddings, L = Same L from Match case
 - Case 2: V = Same V from Match case, L = Text embeddings extracted from "The Great Gatsby" novel
- Hard Non-Match: V and L that are non-matching due to specific *attribute, object, and relation* being *replaced* by incorrect text; hence the overarching context of this text is similar to correct matching text, but only differ by the specific aspect. Thus, more fine-grained understanding is required to discern that it is actually a non-matching text for the image.

Table 3: Replace Task: Statistical Alignment Test Results

Model Pairs	Match				Easy Non-Match				Hard Non-Match			
	CCA (linear/kernel pca)	CKA	SVCCA	CKNNA	CCA	CKA	SVCCA	CKNNA	CCA	CKA	SVCCA	CKNNA
Gemma2 & DINOv2	0.94 / 0.94	0.286	0.432	0.209	0.48 / 0.47	0.020	0.100	0.006	0.94 / 0.94	0.293	0.403	0.194
Llama3.2 & DINOv2	0.94 / 0.95	0.447	0.425	0.275	0.47 / 0.47	0.027	0.097	0.010	0.94 / 0.94	0.431	0.444	0.240
OLMo2 & DINOv2	0.95 / 0.95	0.439	0.505	0.287	0.47 / 0.47	0.028	0.102	0.008	0.95 / 0.95	0.422	0.482	0.252
Gemma2 & ResNet50	0.91 / 0.91	0.25	0.430	0.190	0.51 / 0.51	0.028	0.100	0.006	0.91 / 0.91	0.250	0.420	0.190

B.3.2 Add task

- Match: V and L that are matching (i.e., correct correspondence)
- Easy Non-Match: V and L that have clear non-matching aspects
 - Case 1: V = White-noise image embeddings, L = Same L from Match case
 - Case 2: V = Same V from Match case, L = Text embeddings extracted from "The Great Gatsby" novel
- Hard Non-Match: V and L that are non-matching due to specific *attribute, object* being *added*, which leads to incorrect correspondence of reality

Table 4: Add Task: Statistical Alignment Test Results

Model Pairs	Match				Easy Non-Match				Hard Non-Match			
	CCA	CKA	SVCCA	CKNNA	CCA	CKA	SVCCA	CKNNA	CCA	CKA	SVCCA	CKNNA
Gemma2 & DINOv2	0.95 / 0.94	0.311	0.425	0.201	0.48 / 0.47	0.022	0.095	0.007	0.94 / 0.94	0.323	0.448	0.206
Llama3.2 & DINOv2	0.95 / 0.95	0.451	0.424	0.257	0.48 / 0.48	0.027	0.097	0.011	0.95 / 0.94	0.444	0.447	0.259
OLMo2 & DINOv2	0.94 / 0.94	0.448	0.491	0.269	0.47 / 0.47	0.028	0.097	0.009	0.94 / 0.94	0.448	0.481	0.265
Gemma2 & ResNet50	0.89 / 0.89	0.309	0.420	0.199	0.51 / 0.51	0.028	0.100	0.006	0.89 / 0.89	0.320	0.400	0.200

B.3.3 Swap task

- Match: V and L that are matching (i.e., correct correspondence)
- Easy Non-Match: V and L that have clear non-matching aspects
 - Case 1: V = White-noise image embeddings, L = Same L from Match case
 - Case 2: V = Same V from Match case, L = Text embeddings extracted from "The Great Gatsby" novel
- Hard Non-Match: V and L that are non-matching due to specific *attribute*, *object* being swapped

Table 5: Swap Task: Statistical Alignment Test Results

Model Pairs	Match					Easy Non-Match				Hard Non-Match			
	CCA	CKA	SVCCA	CKNNA		CCA	CKA	SVCCA	CKNNA	CCA	CKA	SVCCA	CKNNA
Gemma2 & DINOv2	0.95 / 0.95	0.317	0.440	0.228		0.47 / 0.47	0.037	0.124	0.014	0.94 / 0.94	0.330	0.433	0.220
Llama3.2 & DINOv2	0.94 / 0.94	0.463	0.473	0.311		0.47 / 0.47	0.049	0.129	0.012	0.94 / 0.94	0.453	0.430	0.287
OLMo2 & DINOv2	0.95 / 0.94	0.466	0.462	0.312		0.480 / 0.48	0.05	0.130	0.012	0.94 / 0.94	0.455	0.511	0.276
Gemma2 & ResNet50	0.89 / 0.89	0.315	0.430	0.221		0.50 / 0.50	0.029	0.123	0.014	0.89 / 0.89	0.315	0.428	0.220

C Spread Loss Formulation for Multimodal Framework

C.1 Our Formulation of Spread Loss for Multimodal Representation Learning

$$\mathcal{L}_{spread} = \frac{1}{2} [\mathcal{L}_{spread-VL} + \mathcal{L}_{LV}] \quad (13)$$

The formulation for VL direction is a weighted sum with α controlling the trade-off between context-level alignment and intra-context contrast:

$$\mathcal{L}_{spread-VL} = (1 - \alpha) \mathcal{L}_{ConCon} + \alpha \mathcal{L}_{contextNCE} \quad (14)$$

For $\alpha \in [0, 1]$, per-sample case of $\mathcal{L}_{spread-VL}$ is defined as:

$$\mathcal{L}_{spread-VL}(\Phi_V, v_i, B) = (1 - \alpha) \mathcal{L}_{ConCon}(\Phi_V, v_i, B) + \alpha \mathcal{L}_{contextNCE}(\Phi_V, v_i, B) \quad (15)$$

where

$$\mathcal{L}_{ConCon}(\Phi_V, v_i, B) = -\frac{1}{|C_L(i)|} \sum_{c_l \in C_L(i)} \log \left(\frac{\sigma(v_i, c_l)}{\sigma(v_i, c_l) + \sum_{\tilde{c}_l \in \tilde{C}_L(i)} \sigma(v_i, \tilde{c}_l)} \right), \quad (16)$$

$$\mathcal{L}_{contextNCE}(\Phi_V, v_i, B) = -\log \left(\frac{\sigma(v_i, l_{P_i})}{\sum_{c_l \in C_L(i)} \sigma(v_i, c_l)} \right). \quad (17)$$

The overall loss $\mathcal{L}_{spread-VL}(\Phi_V, B)$ is computed by averaging over all $(v_i, l_{P_i}, l_{N_i}) \in B$:

$$\mathcal{L}_{spread-VL}(\Phi_V, B) = \frac{1}{|B|} \sum_{i=1}^{|B|} \mathcal{L}_{spread-VL}(\Phi_V, v_i, B).$$

Recall that since hard-negative texts do not have correct corresponding images, the formulation for LV direction omits hard-negative texts. So, it follows the standard LV loss in contrastive learning scheme of CLIP models:

$$\mathcal{L}_{LV}(\Phi_L, B) = -\frac{1}{|B|} \sum_{i=1}^{|B|} \log \frac{\sigma(l_{P_i}, v_i)}{\sum_{j=1, j \neq i}^N \sigma(l_{P_i}, v_j)} \quad (18)$$

C.2 Spread Loss Formulation in Visual Representation Learning Domain

Spread loss in vision representation learning domain [?] is constructed by a weighted sum of a supervised contrastive loss ($\mathcal{L}_{sup}$) [?] and a class-conditional InfoNCE loss ($\mathcal{L}_{cNCE}$) [?]. The core motivation is similar in our multimodal scheme: training encoder to produce representations of the data by pulling together similar points (positive pairs) and pushing apart dissimilar points (negative pairs).

Formally, let B be a batch of data from dataset $\mathcal{D}$. Define the positive set:

$$P(i, B) = \{x^+ \in B \setminus \{x_i\} : h(x^+) = h(x_i)\}$$

and the negative set:

$$N(i, B) = \{x^- \in B \setminus \{x_i\} : h(x^-) \neq h(x_i)\},$$

where $h(x)$ denotes the class label of x and $a(x_i)$ be an augmentation of x_i . Define the similarity (i.e., cosine similarity) with temperature hyperparameter $\tau > 0$:

$$\sigma_f(x, x') = \exp\left(\frac{f(x)^\top f(x')}{\tau}\right).$$

For $\alpha \in [0, 1]$, the per-sample spread loss is defined as:

$$\mathcal{L}_{spread}(f, x_i, B) = (1 - \alpha)\mathcal{L}_{sup}(f, x_i, B) + \alpha\mathcal{L}_{cNCE}(f, x_i, B), \quad (19)$$

where

$$\mathcal{L}_{sup}(f, x_i, B) = -\frac{1}{|P(i, B)|} \sum_{x^+ \in P(i, B)} \log\left(\frac{\sigma_f(x_i, x^+)}{\sigma_f(x_i, x^+) + \sum_{x^- \in N(i, B)} \sigma_f(x_i, x^-)}\right), \quad (20)$$

$$\mathcal{L}_{cNCE}(f, x_i, B) = -\log\left(\frac{\sigma_f(x_i, a(x_i))}{\sum_{x^+ \in P(i, B)} \sigma_f(x_i, x^+)}\right). \quad (21)$$

The overall loss $\mathcal{L}_{spread}(f, B)$ is computed by averaging over all $x_i \in B$:

$$\mathcal{L}_{spread}(f, B) = \frac{1}{|B|} \sum_{x_i \in B} \mathcal{L}_{spread}(f, x_i, B).$$

$\mathcal{L}_{sup}$ encourages intra-class clustering by pulling together same-class samples, while $\mathcal{L}_{cNCE}$ repels within-class samples except for augmentations. Their combination spreads same-class points while preserving instance-level attraction, promoting more structured representation spaces.

C.3 Side-by-Side Comparison

Intuition. $\mathcal{L}_{ConCon}$ pulls an image toward all texts that share its visual context (analogous to class-level clustering, performed by $\mathcal{L}_{sup}$), while $\mathcal{L}_{contextNCE}$ pushes it away from other texts inside that context except its primary caption (analogous to instance discrimination, performed by $\mathcal{L}_{cNCE}$).

Table 6 summarizes the symbols used in the Spread loss in vision domain (left column) and their direct counterparts in our multimodal formulation (right column). The first row clarifies that the visual setting relies on a single encoder $f(\cdot)$, whereas the multimodal variant distinguishes between an image encoder Φ_V and a text encoder Φ_L . Subsequent rows pair up the anchor, positive, and negative sets, making it explicit that class labels in the visual domain translate to visual contexts (collections of captions). Finally, the similarity function retains the same softmax-temperature structure; only the argument types differ (image-image vs. image-text).

Table 7 presents the four loss terms from an objective-driven perspective. Each row identifies the loss and breaks down its optimization goal into four parts: the anchor being updated, the positive samples it should align with, the negatives it should separate from, and the broader motivation behind this push-pull dynamic. This format highlights a consistent analogy: $\mathcal{L}_{sup}$ clusters images by class, while our $\mathcal{L}_{ConCon}$ clusters images with all captions from the same visual context. In contrast, $\mathcal{L}_{cNCE}$ and $\mathcal{L}_{contextNCE}$ act as sharpening losses, refining clusters by contrasting a specific positive — an augmentation for vision case and a primary caption for our multimodal case — against hard negatives.

Table 6: Notation Comparison of Spread Loss in Vision vs. Multimodal Domain (Ours)

Symbol	Vision Spread	Multimodal Spread (Ours)
Encoders	$f(\cdot)$	$\Phi_V(\cdot), \Phi_L(\cdot)$
Anchor	x_i (i.e., image)	$v_i = \Phi_V(x_i)$
Positives	$P(i)$ same class	$C_L(i) = \{l_{P_i}, l_{N_i}\}$ (i.e., similar context texts)
Negatives	$N(i)$ diff. class	$\tilde{C}_L(i) = L \setminus C_L(i)$
Similarity	$\sigma(x, x')$	$\sigma(v, l)$

Table 7: "Objective" view of each loss term.

Loss	Anchor	Positives	Negatives	Goal
$\mathcal{L}_{sup}$	x_i	same class	other classes	intra-class cohesion
$\mathcal{L}_{ConCon}$	v_i	$C_L(i)$	$\tilde{C}_L(i)$	cross-modal cohesion
$\mathcal{L}_{cNCE}$	x_i	$a(x_i)$	other $P(i)$	instance sharpening
$\mathcal{L}_{contextNCE}$	v_i	l_{P_i}	$C_L(i) \setminus \{l_{P_i}\}$	context sharpening

D Model Analysis & Training Configuration

D.1 Pretrained Unimodal Backbones

LM				VM			
Model	Size	# Layers	Dim	Model	Size	# Layers	Dim
Gemma2	2B	26	2304	DINOv2 (SSL)	86M (ViT-B)	12	768
	9B	42	3584		300M (ViT-L)	12	1024
Llama3.2	1B	16	2048	MAE (SSL)	86M	12	768
	3B	32	4096	Swav (SSL)	23M	50	2048
OLMo2	7B	32	4096	Swin (Sup)	88M	50	2048
	13B	40	5120	ViT (Sup)	86M	12	768
				ResNet50 (Sup)	23M	50	2048

Table 8: Configuration of pretrained unimodal models used in experiments. For MAE, we used ViT as backbone architecture. For Swav and Swin, we used ResNet50 as backbone architecture.

D.2 JAM (Joint Autoencoder Modulator) Framework

We experiment JAM (Joint Autoencoder Modulator) framework with 3 hidden layers with dimension size 512, bottleneck/latent layer with dimension size 256, drop out ratio of 0.1, batch size of 32, and SwiGLU activation. With this architecture, Table 9 shows the model analysis of JAM attached to each pretrained unimodal backbones.

With the extracted unimodal features (language and vision, respectively) data, for each task setting, we use 70-15-15 train/validation/test splits. For each task, we train our Joint Autoencoder Modulator (JAM) with all the loss schemes ($\mathcal{L}_{spread}$, $\mathcal{L}_{con}$, $\mathcal{L}_{NegCon}$) for 100 epochs with a batch size of 32, using data seeds 5, 42, and 55. The reported scores are the average of recall scores across different seeds. Both autoencoders are optimized jointly using AdamW [?] with gradient clipping (1.0) and a cosine annealing scheduler. We initialize the logit scaling parameter in log-space as $\log(1/0.07)$, following the common CLIP [?] initialization strategy. During training, the effective scale is recovered via exponentiation, allowing the model to start with sharper similarity distributions and learn an appropriate temperature dynamically. The reconstruction loss is weighted by a linearly

decaying factor $\lambda(t)$, decreasing from 1.0 to 0.1 over training epochs to gradually emphasize the alignment objective. Every five epochs, we compute image-to-text Recall@1 on the validation set, applying early stopping if no improvement is observed for five consecutive validations. We evaluate on two retrieval settings: (1) binary choice between the positive and its hard negative (standard evaluation scheme in fine-grained task setting [? ?]), and (2) a 5-way choice including three additional distractors.

Pretrained Backbone	JAM Parameters (M)	JAM FLOPs (G)
Gemma2 (2B)	11.55	2.39
Gemma2 (9B)	16.20	3.31
Llama3.2 (1B)	10.55	2.06
Llama3.2 (3B)	18.31	3.81
OLMo2 (7B)	18.31	3.81
OLMO2 (13B)	21.82	4.58
DINOv2 (ViT-B)	7.26	1.40
DINOv2 (ViT-L)	8.90	1.72

Table 9: JAM framework analysis attached to pretrained backbones

E Further Results

Language Backbone (Model Size)	Vision Backbone (Model Size)	Alignment Method for JAM	Replace Task		Add Task		Swap Task	
			Recall@1 (binary)	Recall@1 (5-way)	Recall@1 (binary)	Recall@1 (5-way)	Recall@1 (binary)	Recall@1 (5-way)
Gemma2 (2B)	MAE (SSL)	Con	58.55	50.33	56.37	50.46	63.73	49.29
		NegCon	77.62	69.38	89.15	80.16	70.81	58.63
		Spread	89.09	71.16	94.35	84.98	79.05	58.63
Gemma2 (2B)	Swav (SSL)	Con	68.08	62.34	65.25	54.41	62.60	59.21
		NegCon	83.31	76.53	95.20	91.63	80.75	67.04
		Spread	88.88	76.98	95.83	90.64	81.01	67.70
Gemma2 (2B)	Swin (Sup)	Con	64.94	58.17	56.87	50.66	62.91	59.52
		NegCon	85.07	78.35	96.39	91.66	77.93	70.92
		Spread	85.16	79.01	97.87	92.22	80.32	71.46
Gemma2 (2B)	ViT (Sup)	Con	60.78	54.74	60.79	55.05	59.37	52.28
		NegCon	86.06	75.89	96.31	92.50	59.21	49.58
		Spread	86.35	75.26	95.37	89.62	82.31	67.72
Gemma2 (9B)	DINOv2 (ViT-L; 300M)	Con	68.77	63.95	59.31	54.58	60.92	51.85
		NegCon	87.52	84.33	98.89	93.14	84.44	69.69
		Spread	89.66	83.34	98.89	96.30	84.44	73.38
Llama3.2 (1B)	MAE (SSL)	Con	67.82	54.48	61.08	55.33	61.19	54.68
		NegCon	82.92	74.09	94.16	84.80	76.77	68.26
		Spread	86.23	74.96	96.85	92.03	80.47	68.67
Llama3.2 (1B)	Swav (SSL)	Con	65.94	58.97	68.12	62.93	75.39	68.03
		NegCon	92.37	83.91	92.12	87.48	78.62	71.84
		Spread	88.63	79.24	98.94	94.75	83.29	69.99
Llama3.2 (1B)	Swin (Sup)	Con	70.59	63.60	69.70	63.48	75.39	68.03
		NegCon	86.01	83.05	93.89	88.39	69.11	63.45
		Spread	87.24	76.69	94.72	94.16	77.62	68.84
Llama3.2 (1B)	ViT (Sup)	Con	66.81	57.49	64.50	60.70	64.87	59.79
		NegCon	82.81	77.15	93.16	90.57	74.50	63.18
		Spread	87.56	76.13	97.38	92.23	74.23	57.79
Llama3.2 (3B)	DINOv2 (ViT-L)	Con	65.24	59.14	62.38	57.9	73.12	65.18
		NegCon	84.54	82.66	92.61	88.16	75.93	70.27
		Spread	89.43	82.48	95.74	91.25	82.44	76.77
OLMo2 (7B)	MAE (SSL)	Con	66.88	57.37	68.12	56.98	58.36	51.85
		NegCon	86.39	74.57	93.05	88.33	71.94	60.62
		Spread	83.05	77.84	96.39	90.64	79.32	64.52
OLMo2 (7B)	Swav (SSL)	Con	66.38	61.12	67.38	62.58	63.75	58.65
		NegCon	88.18	83.31	92.12	88.50	82.23	69.98
		Spread	93.27	85.63	98.43	93.70	81.01	72.81
OLMo2 (7B)	Swin (Sup)	Con	65.35	59.45	70.82	63.79	55.67	50.85
		NegCon	89.40	86.70	95.28	89.35	73.66	71.96
		Spread	91.32	82.11	97.87	94.72	80.16	70.81
OLMo2 (7B)	ViT (Sup)	Con	70.56	64.79	69.98	58.86	69.15	60.36
		NegCon	84.61	80.77	94.16	87.86	81.74	67.30
		Spread	89.25	81.01	98.43	95.18	76.49	66.44
OLMo2 (13B)	DINOv2 (ViT-L)	Con	65.86	68.68	59.21	62.52	62.28	52.12
		NegCon	89.36	82.68	93.7	91.66	78.89	67.57
		Spread	90.53	83.18	97.96	92.42	84.71	69.69

Table 10: Image-to-Text Retrieval Results of Joint Autoencoder Modulator (JAM) for Vision-Language Compositionality with wider set of pretrained backbones.

F Compute Systems/Resources

All experiments were conducted in either set-ups: Apple Macbook Pro (M2 chip) or NVIDIA RTX 3080. We utilized NVIDIA RTX 3080 system for language models’ feature extraction and M2 chip

system for vision models' feature extraction. Running our JAM framework is done in both set-ups. Finetuning CLIP model is conducted in NVIDIA RTX 3080 system.